

Morse

Theoretical background

We all know how important reliable communication is. Unfortunately, however, advanced solutions sometimes fail, requiring a return to old, simple, and proven methods. Morse code transmission is precisely such a method.

Goal

Your task is to create a transcoder of ASCII string to Morse signal and vice-versa.

You must generate a pure Morse signal in time domain that can be used to modulate HF carrier and directly sent to the other side. It means that a dot '.' represents a single signal 1 and a dash '-' are three ones. 0 – means that there is silence.

In Morse signal, letters are separated by three consecutive zeros (000) and words are separated by seven consecutive zeros (0000000). One zero (0) separates dots and dashes.

Zeros (silence) at the beginning and at the end of the Morse transmission should be ignored.

Morse code signal is sent/received on the lsb when the msb is set on the first byte of the transmission. When msb is cleared, it means the transmission of 7-bit ASCII code. We use capital letters only.

Morse signal is transmitted as one bit (0/1) on the lsb of the data ports (i_data, o_data).

We use 7-bit ASCII code (msb is always 0).

Table 1 Morse Code used in this task

ASCII	Morse	ASCII	Morse
A	.-	N	-. .
B	-... .	O	---
C	-.-.-	P	.-.-.
D	-.. .	Q	--.-
E	.	R	.-.-
F	..-.-	S	
G	--.-	T	-
H		U	..-.
I	.. .	V	...-.
J	.-.-.-	W	.-.-
K	-. -	X	-.-.-
L	.-... .	Y	-. -.-
M	--	Z	--... .
0	-----	.	.-.-.-.-

1	.-----	,	---.---
2	..----	+	.-.-.
3	...--	-	-.....-
4	-	?	..--..
5		_	..--.-
6	-.....	@	.--.-.
7	--...		
8	----..		
9	-----.		

Example

ASCII: SOS A

Morse symbols: ...---... .-

Morse signal: 101010001110111011100010101000000010111

The Morse signal should go on the lsb, one bit on every byte.

Provided module

You can edit only the *task_3.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 2 List of inputs and outputs of the task_3 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet the task receives a sequence of data on the *i_data* port. The maximal amount of data is 4KB. The beginning of the data block is marked by the *i_first* signal and the end by the *i_last* signal. Throughout the data transfer, the *i_valid* signal is high.

Task receives data in the way shown in the figure below:

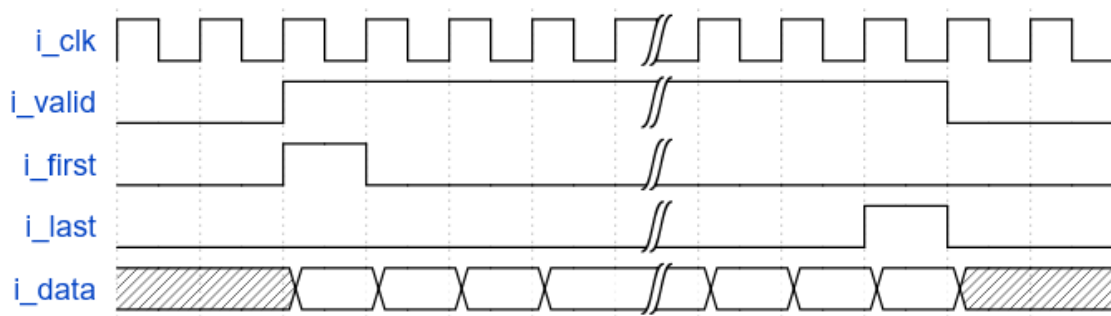


Figure 1 Input waveforms.

To send data from the task in a proper way one needs to provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in output packet.

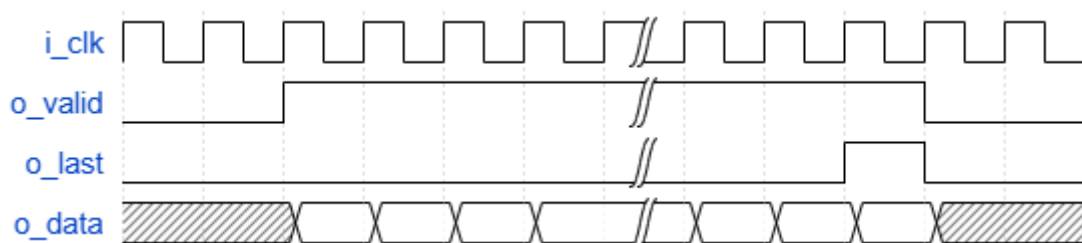


Figure 2 Output waveforms.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The evaluation is done by comparing the sequence returned by the task with the expected sequence calculated by the reference model in the Judge. If the sequences are the same, the task is marked as correctly solved.

In Development mode, the task will be tested using a single set of data that is randomized on each run. In Evaluation mode, the task will be tested using 2 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn 5 points for correctly processing each test vector. In the Development mode, this value simply indicates that the task passed the test. In the Evaluation mode, your total base score is calculated by multiplying the number of points by the number of test vectors (2), so you can earn up to 10 points.

Additional points (up to 5) may be awarded for **resource utilization when latency is below 35µs (3272 cycles)** (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).